

RAG - 3

RAG의 기본 흐름

RAG는 일반적으로 다음 순서로 진행됩니다.

1. **Load**: 문서를 불러온다.
2. **Split**: 문서를 적절한 크기로 나눈다.
3. **Embed**: 문서를 벡터로 변환한다.
4. **Store**: 벡터 저장소에 저장한다.
5. **Retrieve**: 질문과 관련된 문서를 검색한다.
6. **Generate**: 검색 결과를 바탕으로 답변을 생성한다.

이번 시간에는 이 중 **첫 번째 단계인 Load**를 집중적으로 학습합니다.

1. Load 단계란 무엇인가?

Load 단계는 문서를 단순히 여는 것이 아닙니다.

RAG에서 활용할 수 있도록 **문서를 표준화된 형태로 읽어오는 과정**입니다.

사람은 PDF 파일을 열어서 내용을 읽을 수 있지만, 프로그램은 PDF 자체를 바로 이해하지 못합니다.

그래서 먼저 문서 안의 내용을 텍스트로 추출하고, 출처나 페이지 정보 같은 부가 정보도 함께 정리해야 합니다.

이 과정을 거쳐야만 다음 단계인 문서 분할(Split)과 임베딩(Embed)이 가능해집니다.

즉, Load 단계는 다음과 같이 정리할 수 있습니다.

다양한 원본 문서를

LangChain이 다룰 수 있는 **Document** 형식으로 변환하는 과정

2. Document Loader란?

LangChain에서는 문서를 불러오는 도구를 **Document Loader**라고 부릅니다.

Document Loader는 문서의 종류에 따라 적절한 방식으로 내용을 읽고, 이를 LangChain의 `Document` 객체로 바꿔줍니다.

예를 들어,

- `.txt` 파일은 `TextLoader`
- `.pdf` 파일은 `PyMuPDFLoader`
- 웹페이지는 `WebBaseLoader`

와 같이 사용할 수 있습니다.

중요한 점은 원본 데이터의 종류가 달라도, 최종 결과는 모두 같은 형식의 `Document` 객체로 통일된다는 것입니다.

이 덕분에 이후 단계에서는 문서 종류와 관계없이 같은 방식으로 처리할 수 있습니다.

3. Document 객체의 구조

문서를 불러오면 LangChain에서는 보통 `Document` 객체 리스트가 반환됩니다.

이때 각 `Document` 는 크게 두 가지 요소를 가집니다.

3-1. page_content

문서의 실제 내용입니다.

즉, 사람이 읽는 본문 텍스트가 들어 있습니다.

예시:

- 강의안 본문
- 논문 내용
- 웹페이지 설명문
- txt 파일의 텍스트

3-2. metadata

문서에 대한 부가 정보입니다.

예시:

- 파일 이름
- 파일 경로
- 페이지 번호

- 문서 출처
- URL
- 작성자 정보

따라서 **Document** 는 단순한 문자열이 아니라,
본문과 출처를 함께 보관하는 데이터 구조라고 이해하면 됩니다.

4. metadata가 중요한 이유

초기에는 본문 내용만 중요해 보일 수 있지만, 실제 RAG에서는 metadata가 매우 중요합니다.

예를 들어,

- 어떤 PDF의 **몇 페이지**에서 가져온 내용인지 표시할 수 있고,
- 어떤 **파일**에서 나온 문장인지 구분할 수 있으며,
- 특정 문서만 검색 대상으로 제한할 수도 있습니다.

즉, metadata는 단순 참고용 정보가 아니라,
검색 품질과 신뢰도를 높이는 핵심 요소입니다.

5. 주요 로더 종류

LangChain에는 다양한 로더가 있지만, 입문 단계에서는 아래 로더들을 먼저 익히는 것이 좋습니다.

5-1. TextLoader

텍스트 파일(**.txt**)을 읽을 때 사용합니다.

가장 단순한 로더이며, 학습자에게 Document 구조를 처음 보여줄 때 적합합니다.

5-2. PyMuPDFLoader

PDF 문서를 불러올 때 많이 사용합니다.

RAG 실습에서 자주 활용되며, 페이지 단위로 처리하기에 좋습니다.

5-3. WebBaseLoader

웹페이지를 문서처럼 읽어올 때 사용합니다.

공지사항 페이지, 블로그 글, 설명 페이지 등을 가져올 수 있습니다.

5-4. CSVLoader

CSV 형식 데이터를 텍스트 기반 문서처럼 불러올 때 사용합니다.

표 데이터를 다루는 기초 예제로 활용할 수 있습니다.

5-5. DoclingLoader

PDF, DOCX, PPTX처럼 구조가 복잡한 문서를 보다 풍부하게 파싱할 때 적합합니다.

표나 레이아웃이 중요한 문서에서 유용합니다.

6. `load()` 와 `lazy_load()`

문서 로더는 보통 두 가지 방식으로 문서를 불러옵니다.

6-1. `load()`

전체 문서를 한 번에 모두 읽습니다.

장점:

- 사용하기 쉽다.
- 코드가 단순하다.
- 소규모 실습에 적합하다.

단점:

- 문서 양이 많으면 메모리를 많이 사용할 수 있다.

6-2. `lazy_load()`

문서를 하나씩 순차적으로 불러옵니다.

장점:

- 대용량 문서 처리에 유리하다.
- 메모리 부담을 줄일 수 있다.

단점:

- 입문자에게는 다소 낯설 수 있다.

쉽게 말하면,

- `load()` 는 자료를 한 번에 전부 가져오는 방식
- `lazy_load()` 는 필요할 때마다 하나씩 가져오는 방식

이라고 이해하면 됩니다.

7. 텍스트 파일 불러오기 예제

다음은 가장 기본적인 텍스트 파일 로딩 예제입니다.

```
from langchain_community.document_loaders import TextLoader

loader=TextLoader("sample.txt",encoding="utf-8")
docs=loader.load()

print(len(docs))
print(docs[0].page_content[:300])
print(docs[0].metadata)
```

코드 해설

- `TextLoader("sample.txt")` `sample.txt` 파일을 읽기 위한 로더를 생성합니다.
- `loader.load()`
문서를 읽어서 `Document` 리스트 형태로 반환합니다.
- `len(docs)`
불러온 문서 개수를 확인합니다.
- `docs[0].page_content`
첫 번째 문서의 본문 일부를 확인합니다.
- `docs[0].metadata`
첫 번째 문서의 메타데이터를 확인합니다.

이 예제를 통해 학습자는 “문서를 불러오면 어떤 결과가 나오는지”를 가장 쉽게 확인할 수 있습니다.

8. PDF 불러오기 예제

다음은 PDF 문서를 불러오는 예제입니다.

```
from langchain_community.document_loaders import PyMuPDFLoader

loader=PyMuPDFLoader("lecture_notes.pdf")
```

```
docs=loader.load()

print(f"문서 수:{len(docs)}")
print(docs[0].metadata)
print(docs[0].page_content[:500])
```

코드 해설

- `PyMuPDFLoader("lecture_notes.pdf")`
PDF 파일을 읽는 로더를 생성합니다.
- `docs = loader.load()`
PDF를 읽어 `Document` 리스트로 반환합니다.
- 페이지 단위로 읽히는 경우, 각 페이지가 하나의 문서처럼 저장될 수 있습니다.
- `docs[0].metadata`
첫 번째 페이지에 대한 정보가 담길 수 있습니다.
- `docs[0].page_content`
첫 번째 페이지의 본문 텍스트를 확인합니다.

9. PDF를 하나의 흐름으로 불러오기

경우에 따라서는 PDF를 페이지별로 나누지 않고, 하나의 긴 텍스트로 읽고 싶을 수도 있습니다.

```
from langchain_community.document_loaders import PyMuPDFLoader

loader=PyMuPDFLoader(
    "lecture_notes.pdf",
    mode="single",
    pages_delimiter="\n--- PAGE BREAK ---\n"
)

docs=loader.load()

print(len(docs))
```

```
print(docs[0].page_content[:1000])
print(docs[0].metadata)
```

코드 해설

- `mode="single"`
전체 PDF를 하나의 문서로 읽습니다.
- `pages_delimiter`
페이지가 바뀌는 위치를 텍스트 구분자로 남겨둘 수 있습니다.

이 방식은 문단의 흐름을 자연스럽게 유지하고 싶을 때 유용합니다.

10. 웹페이지 불러오기 예제

웹페이지 역시 RAG의 문서가 될 수 있습니다.

```
from langchain_community.document_loaders import WebBaseLoader

loader=WebBaseLoader("https://example.com")
docs=loader.load()

print(len(docs))
print(docs[0].page_content[:500])
print(docs[0].metadata)
```

코드 해설

- `WebBaseLoader` 는 웹페이지 내용을 읽어옵니다.
- 블로그, 공지사항, 안내 페이지 등을 지식 문서처럼 다룰 수 있습니다.
- 웹 기반 자료를 RAG에 연결할 때 자주 활용됩니다.

11. PDF 로딩 시 주의할 점

PDF는 실제 수업과 실무에서 가장 많이 활용되지만, 동시에 가장 문제가 많이 생기는 형식이기도 합니다.

다음과 같은 문제가 발생할 수 있습니다.

- 줄바꿈이 이상하게 추출됨
- 문장이 페이지 중간에서 끊김
- 표가 깨짐
- 이미지 중심 PDF는 텍스트가 거의 추출되지 않음

따라서 PDF를 로딩할 때는 단순히 코드가 실행되는지만 볼 것이 아니라, 실제로 텍스트가 잘 읽혔는지 확인하는 습관이 중요합니다.

12. 페이지 단위 로딩과 전체 로딩 비교

PDF를 읽을 때는 보통 두 가지 방식을 비교해볼 수 있습니다.

페이지 단위 로딩

장점:

- 몇 페이지 내용인지 알기 쉽다.
- 출처를 표시하기 좋다.
- 페이지별 검색 결과 확인이 편하다.

단점:

- 문장이 페이지 경계에서 끊길 수 있다.

전체 문서 로딩

장점:

- 문장 흐름이 자연스럽다.
- 문단 맥락을 유지하기 좋다.

단점:

- 페이지 추적이 어렵다.
- 이후 문서 분할이 더 중요해진다.

어떤 방식이 더 좋은지는 문서와 목적에 따라 달라집니다.

DoclingLoader 사용법